We index the book with the two fields without mentioning the `title_synopsis` field in the document. How does it get indexed?

During the indexing process, Elasticsearch fills this third field (`title_synopsis`) by combining the `title` and `synopsis` fields (remember the `copy_to` attribute on these fields?). Thus, a combined field called `title_synopsis` holds the full data from the other fields.

Because we enhanced the schema with this `copy_to` functionality, we can rewrite the search query to become a simple `match` query rather than a `multi-match`.

Listing 15.4 A `match` query against the `title_synopsis` field

```
GET programming_books2/_search
{
  "query": {
    "match": {
      "title_synopsis": {
        "query": "Elasticsearch hands-on example driven",
        "operator": "OR"
      }
    }
  }
}
```

Specifies the title_synopsis field for the match query

We now use a simple `match` query (as opposed to the expensive `multi_match` query described in listing 15.1) to fetch our results. Because the grunt work of copying data to a combined field was done at the indexing time, the search simply queries against this already-available field.

Searching through multiple fields using `multi_match` (or even a `query_string` query, as discussed in chapter 9) is an expensive operation. Instead, we used the combined fields method with the `copy_to` construct. Matching a few fields rather than searching across dozens of them is a good candidate for performance enhancement.

15.1.3 Choosing keyword types over text types

Full-text searches undergo a text analysis phase: they are normalized and tokenized just like indexing, before the results are fetched. This is a computing-intensive operation, and we can avoid it on fields that are the `keyword` type. Keyword fields do not undergo text analysis, thus saving us time and effort when searching.

Consider declaring a `keyword` type for any `text` field if the use case allows searching on keywords. For example, we can create a book with a title defined as a `text` data type as well as a `keyword` field using the multi-fields feature.

Listing 15.5 A multi-field query using the `keyword` type

```
PUT programming_books3
{
  "mappings": {
    "properties": {
```